# JAVA SWING BASED GUI EXPENSE MANAGER

Java Assignment

*Faculty Name*

***DR. VINAY KUMAR SAINI***

*Prepared By*

***SUTANU KUMAR DAS***

kumarsutanudas092005@gmail.com

==========

# *PROJECT ASSIGNMENT REPORT*

## *JAVA SWING BASED GUI EXPENSE MANAGER*

***STUDENT NAME: Sutanu Kumar Das***

***ENROLLMENT NO: 03914815624***

SYSTEM: Linux/OpenJDK 26/Maven/SQLite

[ INITIALIZING REPORT GENERATION... ]

## *1.0 PROJECT ABSTRACT*

The 'Java Swing Based GUI Expense Manager' is a comprehensive solution for personal finance management. In an era where digital transactions are ubiquitous, keeping track of shared expenses, split bills, and debt settlements is a complex task. This application provides a streamlined, user-friendly interface to log expenses, categorize them, and visualize spending habits through dynamic charts. The core utility lies in its 'Settlement Algorithm', which mathematically resolves debts among multiple users with the minimum
number of transactions.

.....

## *2.0 CORE OBJECTIVES*

*Provide a modern Material Design GUI for Java Desktop Applications.
*Implement robust data persistence using Zero-Config SQLite.
*Automate complex debt calculations (Settlements).
*Generate professional PDF and CSV reports for financial auditing.
*Facilitate direct email communication for report sharing.

.....

## *3.0 SYSTEM ARCHITECTURE (MVC)*

Theapplication follows the Model-View-Controller pattern:

A.MODEL: Represents the data structures. Includes User, Category, Expense,and Settlement classes. It ensures data integrity before it reaches the database.

B.VIEW: The graphical interface. Built using Java Swing and enhanced by FlatLaf. It provides separate panels for Dashboard (visuals) and Expense Management (CRUD).

C.CONTROLLER: The bridge. Manages the flow of data between the GUI and the DAO (Data Access Objects). This is where the business logic, such as the splitting algorithm, resides.

.....

## *4.0 DATABASE SCHEMA*

The system uses four interconnected tables in SQLite:
1. users: Stores member profiles (Name, Email).
2. categories: Pre-defined buckets like Food, Travel, etc.
3. expenses: Central log of total amounts paid.
4. expense_splits: Detailed breakdown of who owes how much for each expense.

.....

## *5.0 UTILITY: DatabaseHelper.java*

Amajor technical hurdle in desktop apps is DB setup. I solved this by implementing a dynamic path resolution logic:

CODE LOGIC:
String APP_DIR = System.getProperty("user.home") + File.separator + ".expense-manager";
This ensures the database file is always stored in the user's home directory (~/.expense-manager/), making the application portable across different machines without losing data.

UTILITY: Users don't need to install SQL servers. The app creates its own environment on first run.

.....

## *6.0 LOGIC: The Settlement Algorithm*

The most innovative part of the code is the debt resolution logic. It uses a greedy matching strategy:
1. Calculate net balance for every user (Total Paid - Total Owed).
2. Separate users into 'Debtors' (negative balance) and 'Creditors' (positive balance).
3. Repeatedly match the largest debtor with the largest creditor until balances reach zero.

UTILITY: This prevents 'round-robin' payments and simplifies a group's debt into the fewest possible payments.

.....

## *7.0 PACKAGING & DISTRIBUTION*

Tomake this a 'Desktop Application' rather than just a code project, I performed the following:

7.1 SHADING (Uber-JAR Creation):
I used the maven-shade-plugin to package all dependencies (SQLite drivers, JFreeChart, iText) into a single .jar file. I had to specifically exclude JAR signatures:
<exclude>META-INF/*.SF</exclude>
<exclude>META-INF/*.DSA</exclude>
This prevents 'SecurityException' when running the packaged app.

7.2 SYSTEM INTEGRATION:
For Linux, I created a 'launch.sh' script and an 'ExpenseManager.desktop' entry. This allows the app to appear in the system's Application Menu, just like Firefox or VLC.

.....

## *8.0 HOW TO PACK YOUR OWN JAR*

Ifyou are using the terminal:
1.Ensure your pom.xml has the maven-shade-plugin configured.
2.Run: mvn clean package
3.Find the JAR in the /target folder.

Ifusing IntelliJ:
1.Go to Project Structure -> Artifacts -> Jar -> From modules with dependencies.
2. Build -> Build Artifacts.

.....

## *9.0 DEPLOYMENT TO GITHUB*

Toshare this application so anyone can use it, follow these 'Production-Ready' steps:

9.1REPOSITORY SETUP:
1.Create a .gitignore to exclude /target and database files.
2.git init && git add . && git commit -m 'Initial release'.
3.git push to your remote GitHub repo.

9.2THE 'RELEASES' FEATURE (Critical):
DoNOT just upload code. Use the 'Releases' tab on GitHub:
1.Create a New Release (e.g., v1.0.0).
2.Upload your 'expense-manager-1.0-SNAPSHOT.jar' as a binary asset.

9.3USER INSTRUCTIONS:
Inyour README.md, tell users to:
a)Download the JAR from the Releases tab.
b)Install Java 17 or higher.
c)Run 'java -jar expense-manager.jar'.

.....

## *10.0 CONCLUSION*

This project demonstrates the power of Java for building cross-platform productivity tools. By combining Swing's flexibility with modern librarieslike FlatLaf and SQLite, we have created a tool that is both visually appealingand functionally robust. The project is fully ready for deployment as a standard desktop application.

.....

***[ END OF REPORT ]***